PRÁCTICA 2

DETECCIÓN Y ANÁLISIS DE TRAMAS MEDIANTE WIRESHARK.

Prácticas de Fundamentos de Redes y Comunicaciones – Curso 2025/2026

Jesús Manuel Calle Cancho (jesuscalle@unex.es)
Julio Domínguez Arjona (julioda@unex.es)

Índice

1. Objetivos.
2. Uso de Wireshark.
3. Ejemplo de envío y captura de paquetes.
   3.1. Captura de un paquete o trama mediante Wireshark.
   3.2. Envío de una trama mediante Visual Studio Code.
4. Información de tarjeta de red (ifconfig).
5. Explicación de las funciones de la librería para gestión de interfaces, envío y recepción.
   5.1. Envío de caracteres.
   5.2. Recepción de caracteres.
6. Funciones kbhit y getch.
7. **Tarea a realizar.**
8. **Salida en pantalla.**
9. **Cómo ejecutar la práctica.**
10. Entrega de la sesión práctica.

1. Objetivos.

El principal objetivo de esta entrega es implementar y probar el envío y recepción de caracteres organizados en tramas ethernet a través de las interfaces del equipo. Adicionalmente, se usará el Wireshark para capturar paquetes y analizar las tramas ethernet recibidas, comprobando la información que posee cada uno de los campos que la forman.

2. Uso de Wireshark.

Wireshark es un analizador de protocolos *open-source* que permite inspeccionar protocolos de capa física, de enlace, protocolos de red, capa de transporte y también de capa de aplicación. Mediante Wireshark podemos realizar capturas de paquetes (entrantes y salientes de la tarjeta de red cableada o inalámbrica) en tiempo real y analizarlos posteriormente.

Wireshark implementa una amplia gama de filtros que facilitan diferentes criterios de búsqueda para todos los protocolos soportados, a través de una interfaz sencilla que desglosa por capas cada uno de los paquetes capturados. Igualmente, proporciona visualización de los campos de cada una de las cabeceras y capas que componen los paquetes.

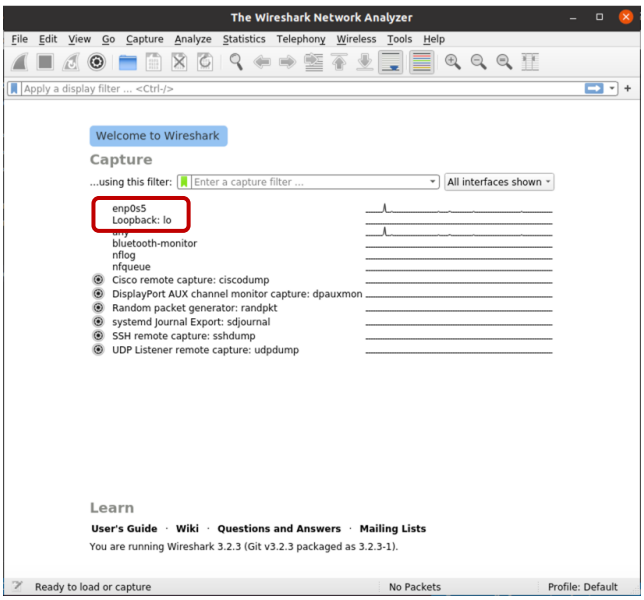


Figura 1: Entorno de Wireshark. Interfaces que se usarán.

Para comenzar a utilizar la aplicación, seleccionaremos la tarjeta de red sobre la que se van a capturar los paquetes entrantes o salientes (figura 1). Para las prácticas usaremos la interfaz correspondiente de la propia tarjeta de red (en la figura 1 viene especificada como **enp0s5)** y la interfaz **loopback**. Ésta última, es una interfaz virtual y no existe físicamente en el equipo, pero realiza todas las funciones de una interfaz normal: siempre está activa, los paquetes destinados a esta interfaz se procesan localmente, los paquetes que se envíen desde esta interfaz a otro destino que no sea la propia interfaz se descartarán sin causar ningún error y no se permite recibir tráfico a través de esta interfaz.

Si no usamos ningún filtro, se muestran todos los paquetes capturados (figura 2). En caso de usar alguno, el filtro no bloqueará la captura de tráfico, solo bloqueará la visibilidad de los paquetes.

Para las prácticas usaremos las opciones principales de Wireshark: comenzar a capturar, dejar de capturar y resetar la captura.

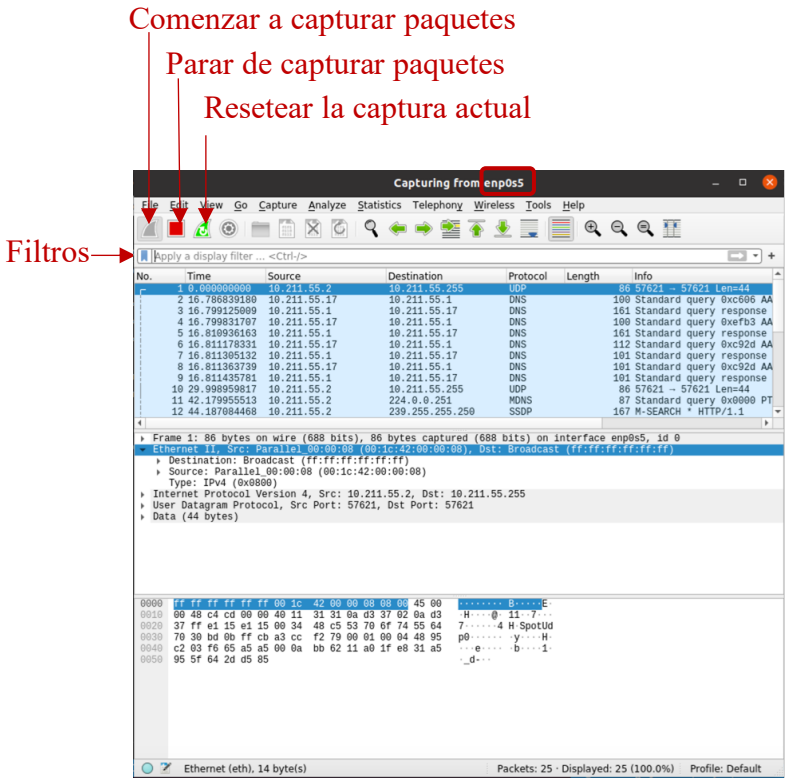


Figura 2: Opciones básicas para capturar paquetes a través de la interfaz enp0s5.

3. Ejemplo de envío y captura de paquetes.

En este apartado vamos a mostrar como enviar una trama que contiene un solo carácter, a través de un programa implementado en C/C++ y lo capturaremos mediante Wireshark a través de la interfaz **loopback**.

3.1. Captura de un paquete o trama mediante Wireshark.

Para capturar paquetes, usaremos Wireshark. Ejecutaremos la aplicación en modo superusuario escribiendo en el terminal **sudo wireshark.** Elegiremos como interfaz **loopback**. Mantendremos el programa abierto para poder capturar paquetes (figura 3). Una vez hayamos enviado el que nosotros deseamos con Visual Studio, analizaremos el paquete capturado.

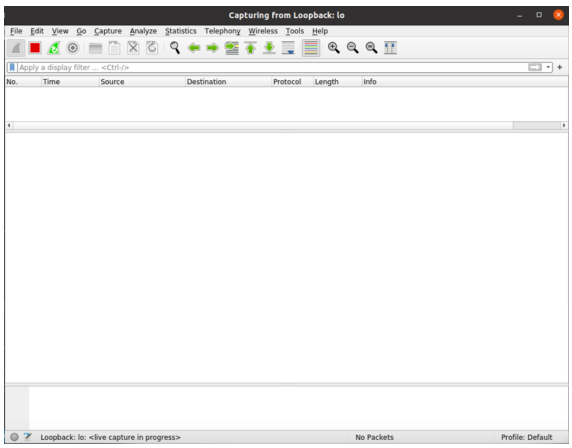


Figura 3: Wireshark listo para capturar paquetes.

3.2. Envío de una trama mediante Visual Studio Code.

Mediante el siguiente programa (Figura 4) enviaremos una trama ethernet que contiene como datos solo el carácter 'M', usando como MAC origen 00:00:00:00:00:00 (mac de **loopback**) y MAC destino 00:01:02:03:04:05. Como tipo de protocolo utilizaremos uno inventado por nosotros, 0x3000.

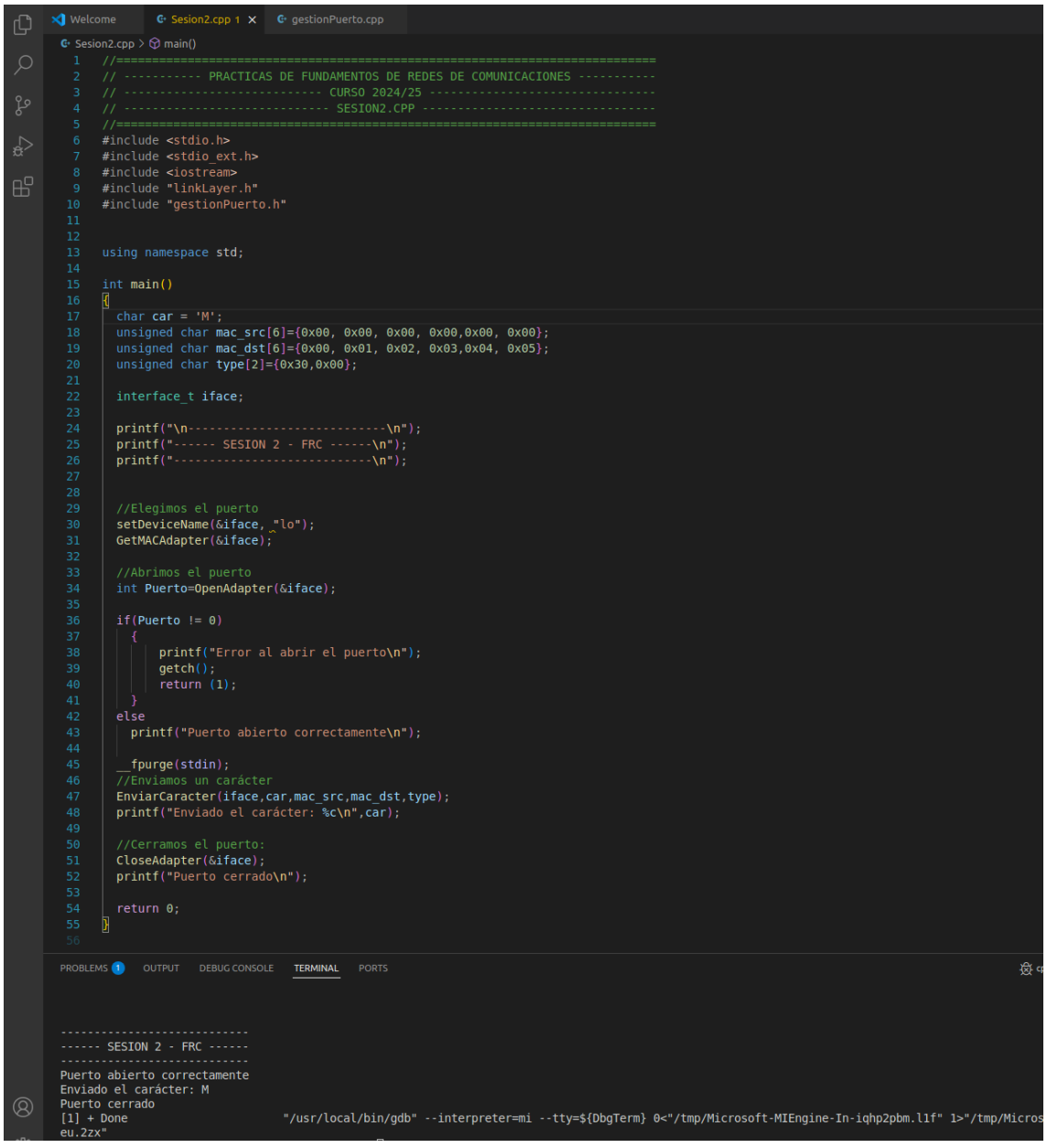


Figura 4: Programa que envía una trama que contiene como datos el carácter 'M'.

Cuando ejecutemos el programa anterior, en wireshark comprobaremos que habremos capturado el paquete deseado, por lo que detendremos la captura de más paquetes para visualizar solo el paquete enviado por nosotros. La información que se reflejará en dicha aplicación se muestra en la figura 5. En dicha figura también se describen los paneles en los que se divide wireshark, así como dónde se muestran los diferentes campos de los que consta un paquete ethernet capturado.

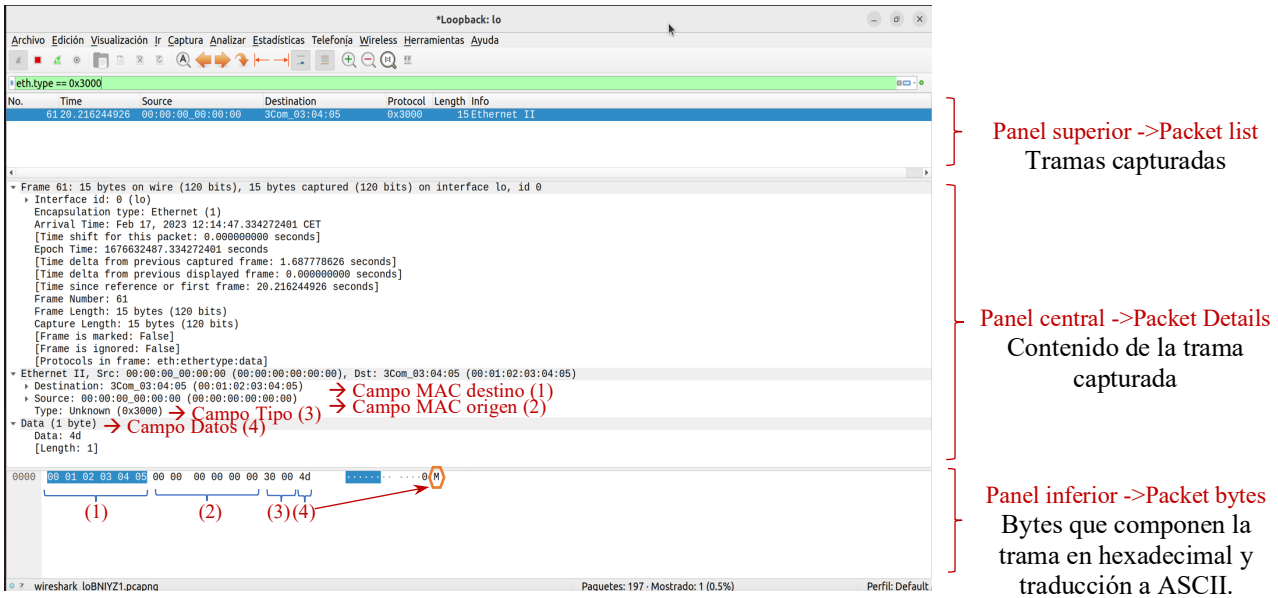


Figura 5: Campos de 1 paquete capturado mediante loopback cuyo dato contiene el carácter 'M'.

4. Información de la tarjeta de red mediante ifconfig.

El comando **ifconfig** permite configurar interfaces de red de un dispositivo, al igual que mostrar información de dichas interfaces usando dicho comando sin parámetros (figura 6). La información que nos interesa a nosotros es la correspondiente al nivel de enlace, es decir, la correspondiente a ethernet. En la figura 6 se muestran bordeadas de un marco rojo, la **mtu** que es la unidad máxima de transferencia, (para ethernet es 1500 bytes) (1), la dirección física (MAC) de una interfaz de red (2), la velocidad de ethernet (en este caso 1 Gb) (3), los paquetes recibidos en bytes (4) y los paquetes enviados en bytes (5).

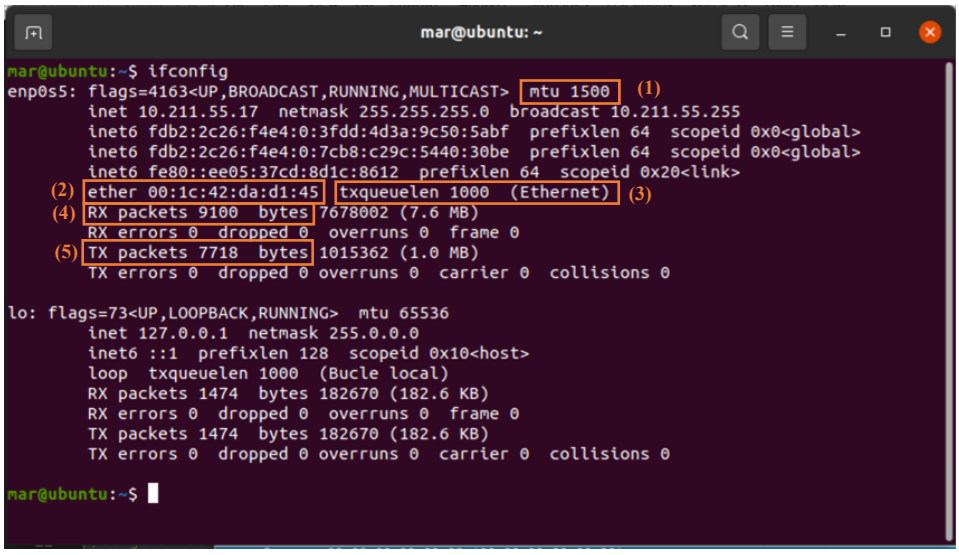


Figura 6: Información de tarjetas de red mediante ifconfig.

5. Explicación de las funciones de la librería para gestión de interfaces, envío y recepción.

5.1. Envío de caracteres.

Para el envío de caracteres haremos uso de la librería **linkLayer.h**. A continuación, se muestra el pseudocódigo de cómo podría ser la función **EnviarCaracter**.

```
void EnviarCaracter (interface, datos, mac_origen, mac_destino, tipo)
{
        ReservarMemoriaDatos;
        AlmacenarDatos;
        ConstruirTrama;
        EnviarTrama;
        LiberarMemoriaDatos;
}
```

5.2. Recepción de caracteres.

Igualmente, para la recepción de caracteres haremos uso de la librería **linkLayer.h**. A continuación, se muestra el pseudocódigo de cómo podría ser la función **RecibirCaracter**.

```
char RecibirCaracter (interface)
{
        Trama=RecibirTrama;
        QuedarseCampoCompletoDatosTrama;
        //Acordaos, el campo datos contiene la mac destino, mac origen, tipo y los
        datos propiamente dichos (caracteres) en este orden.
        Si (hay algo en el campo datos)
          QuedarseDatosRecibidos;  //Quedarse con los caracteres recibidos.
          DevolverDato;
        Sino
          DevolverValor0;
}
```

6. Funciones kbhit() y getch().

La función **kbhit()** permite detectar la pulsación de una tecla. En caso de pulsar una tecla, dicha función la imprime en pantalla y devuelve un valor distinto de 0. Si no se pulsa ninguna, devolvería 0.

La función **getch()** permite leer una tecla pulsada (por ejemplo, la tecla pulsada y detectada por kbhit()) y la almacenaría en una variable de tipo carácter. La tecla leída no se mostraría en pantalla.

7. Tarea a realizar.

Partiendo de la práctica anterior, desarrollar un programa que envíe cualquier carácter que el usuario escriba por teclado. Tanto en el emisor, como en el receptor se mostrará el carácter enviado y recibido. El programa finalizará solo cuando el usuario pulse la tecla ESC. Igualmente, se deberá poder recibir cualquier carácter que llegue al buffer a través de la interfaz, y éste se mostrará por pantalla. La MAC destino, en esta práctica, será codificada en el programa principal. Para obtener la MAC de origen, se debe hacer uso del código desarrollado en la sesión 1. En pantalla se deberá mostrar tanto la MAC origen, como la MAC destino.

8. Salida en pantalla.

La salida en pantalla (figura 7) de la tarea a realizar sería similar a la mostrada a continuación; dependerá de las interfaces disponibles de cada equipo.

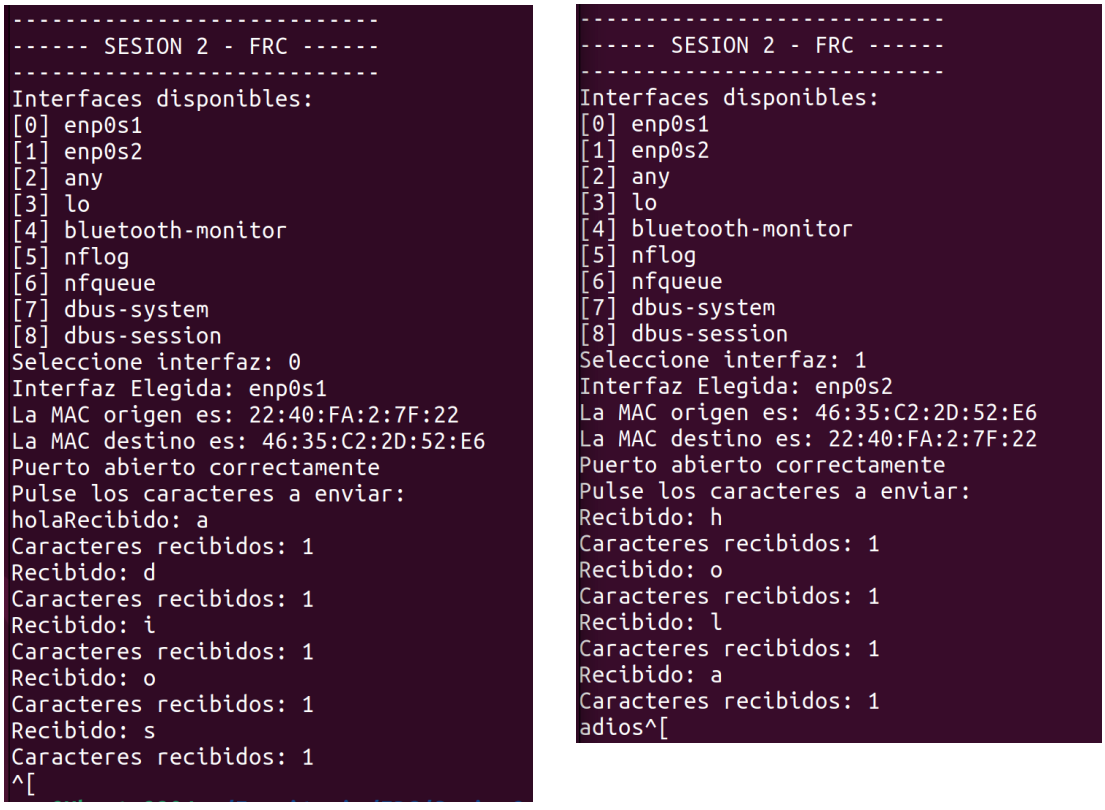


Figura 7: Resultado de la ejecución mediante dos terminales.

En caso de no poder probar con dos interfaces, para esta entrega se podría dar por correcto usar la interfaz virtual loopback para poder enviar y recibir en el mismo equipo, pero hay que tener en cuenta que es necesario usar dos interfaces de red para la siguiente entrega.

La salida sería similar a la siguiente (figura 8):

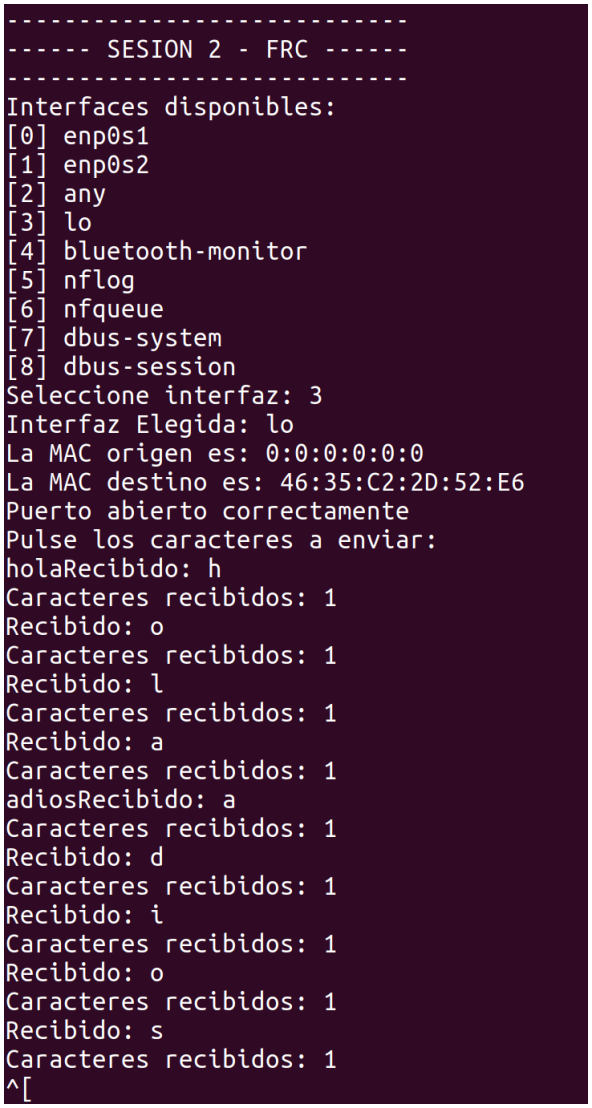


Figura 8: Resultado de la ejecución mediante la interfaz virtual loopback.

9. Cómo ejecutar la práctica.

La práctica se puede ejecutar desde el Visual Studio o directamente desde el Terminal, tal y como se muestra a continuación:

- Visual Studio Code: Opción ***Run/Run Without Debuggin***
- Terminal: ***sudo ./Sesion2*** (Previamente debemos habernos introducido en la carpeta donde se encuentra el fichero ejecutable).

Lo ideal será probar la práctica usando dos equipos conectados a la red local, pero también podremos comprobar su funcionamiento usando un solo equipo. Para ello será preciso abrir dos instancias de la aplicación, bien desde el Visual Studio o bien desde el terminal.

10. Entrega de la sesión práctica.

A través de la herramienta AVUEx, según las instrucciones dadas en clase a este respecto, debe entregarse un archivo comprimido en formato ZIP o RAR que contenga lo siguiente:

- Un archivo AUTORES.TXT, que incluya nombre y apellidos, por este orden, de los autores de la práctica, así como el grupo al que pertenecen ambos.
- Los archivos fuente de la práctica. **Cada fichero fuente** debe incluir **obligatoriamente** el nombre, apellidos y grupo de los autores de la práctica.
- El fichero ejecutable (compilado a partir de los ficheros fuentes entregados) de la práctica.
- La carpeta oculta **.vscode**.
- Debe incorporarse documentación interna adecuada y suficiente como para seguir adecuadamente el código.
- La fecha tope para subir esta ***entrega 2*** se comentará en clase. **Cualquier práctica entregada con posterioridad a la fecha y hora indicada se considerará no válida a todos los efectos**.